

Batch Code: ANP_D1838

Student ID: AF05202193

Student Name: S. Vaishnavi

LAB-07

Questions:

1. What is exception Handling & Its types
 2. Difference between Checked Exception and UnChecked Exception
 3. Write 2 example program for Checked Exception
 4. Write 2 example program for UnChecked Exception
-

1. EXCEPTION HANDLING AND TYPES:

Exception Handling in Java is a mechanism to handle runtime errors, ensuring the normal flow of the program. It prevents abnormal termination of programs. It provides meaningful error messages to users.

It uses five key keywords:

- try - Defines a block of code to test for exceptions.
- catch - Defines a block of code to handle the exception.
- finally - Defines a block of code that always executes (cleanup code).
- throw - Used to explicitly throw an exception.
- throws - Declares exceptions that a method might throw

Types:

- Checked Exceptions (Compile-time exceptions)
 - Verified at compile-time.
 - Must be handled using try-catch or declared with throws.
 - Examples: IOException, SQLException, ClassNotFoundException.
- Unchecked Exceptions (Runtime exceptions)

- Occur during program execution.
 - Not checked at compile-time.
 - Examples: `NullPointerException`, `ArithmeticException`, `ArrayIndexOutOfBoundsException`.
- Errors
 - Serious problems that cannot be recovered by programs.
 - Examples: `OutOfMemoryError`, `StackOverflowError`.
-

2. DIFFERENCE BETWEEN CHECKED EXCEPTION AND UNCHECKED EXCEPTION

FEATURE	CHECKED EXCEPTION	UNCHECKED EXCEPTION
Compile-time checking	Compiler checks them at compile-time. If not handled, program won't compile.	Compiler does not check them. They appear only at runtime.
Handling requirement	Must be handled using try-catch or declared with throws.	Handling is optional. Program compiles even if not handled.
Nature of problem	Represent recoverable conditions (like file not found, database error).	Represent programming mistakes/bugs (like dividing by zero, null access).
Program flow impact	Forces programmer to write defensive code and anticipate failures.	May cause program to crash at runtime if not handled.
Use cases	Used when program can recover gracefully (e.g., ask user to choose another file).	Used when error is due to faulty logic that needs code correction.
Examples	<code>IOException</code> , <code>SQLException</code> , <code>ClassNotFoundException</code> , <code>FileNotFoundException</code> .	<code>NullPointerException</code> , <code>ArithmeticException</code> , <code>ArrayIndexOutOfBoundsException</code> .

3. CHECKED EXCEPTION:

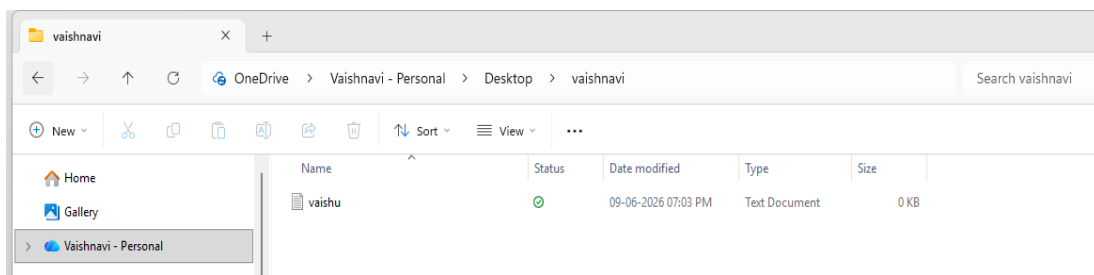
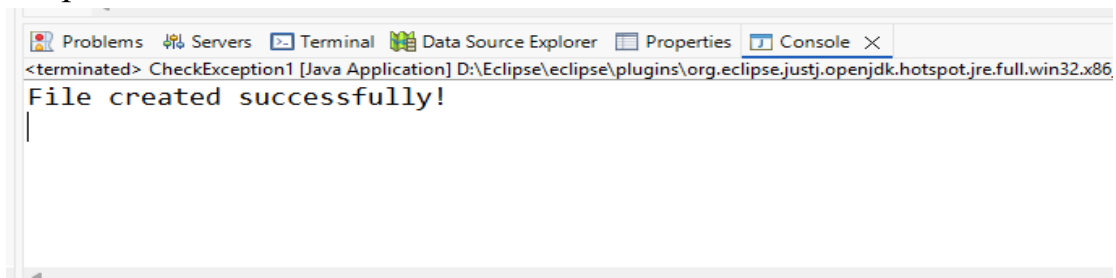
PROGRAM 1

Checked Exception using File Creation

```
import java.io.File;
import java.io.IOException;

public class CheckException1 {
    public static void main(String[] args) {
        File file = new
File("C:\\Users\\acer\\OneDrive\\Desktop\\vaishnavi\\vaishu.txt");
        try {
            if(file.createNewFile()) {
                System.out.println("File created successfully!");
            } else {
                System.out.println("File already exists.");
            }
        } catch (IOException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

Output:



1. Import Statements

- `import java.io.File;` - allows you to work with files.
- `import java.io.IOException;` - needed because `createNewFile()` can throw a Checked Exception.

2. Class Declaration

- `public class CheckedException1` - defines your program class.
- File name must match class name (`CheckedException1.java`).

3. Main Method

- `public static void main (String[] args)` - entry point of the program.

4. File Object Creation

- `File file = new File("C:\\Users\\acer\\OneDrive\\Desktop\\vaishnavi\\vaishu.txt");`
- This doesn't create a file yet. It just creates a File object pointing to that path.

5. `createNewFile()` Method

- `file.createNewFile()` tries to create a new file at that location.
- It returns:
 - `true` → if a new file was created.
 - `false` → if the file already exists.

6. Checked Exception Handling

- Since `createNewFile()` can throw an `IOException`, it must handle with try-catch.
- If something goes wrong (like invalid path, no permission, disk full), the catch block runs.

7. Output

- If file is newly created → prints "File Opened".
 - If file already exists → nothing prints (because your code only prints inside the if).
 - If error occurs → stack trace prints.
-

PROGRAM 2

CHECKED EXCEPTION (VOTING ELIGIBILITY)

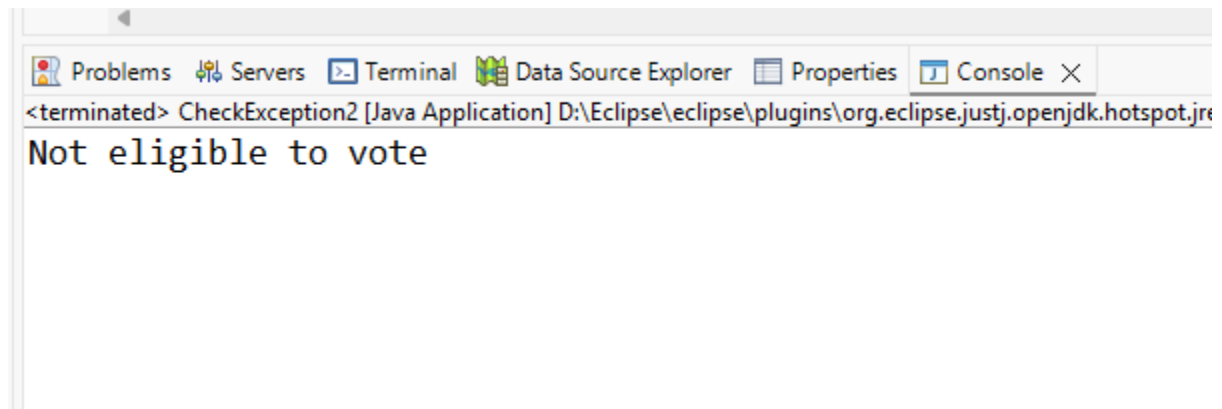
```
import java.io.IOException;
class CheckException2 {
    public static void main(String[] args) {
```

```

int age = 15;
try
{
    if (age < 18)
    {
        throw new IOException("Not eligible to vote");
    }
    else
    {
        System.out.println("Eligible to vote");
    }
}
catch (IOException e)
{
    System.out.println(e.getMessage());
}
}

```

Output:



Import Statement

- `import java.io.IOException;` - brings in the `IOException` class, which is a Checked Exception.

Class & Main Method

- `class CheckException2` - defines your program.
- `public static void main(String[] args)` - entry point of the program.

Variable

- `int age = 15;` - sets the age to 15.

try block

- Inside try, you check if `age < 18`.
- Since 15 is less than 18, you manually throw a new `IOException` with the message "Not eligible to vote".
- This is how you force a Checked Exception when a condition fails.

catch block

- The thrown exception is caught by the catch (`IOException e`) block.
- `System.out.println(e.getMessage());` prints the message stored in the exception - "Not eligible to vote".

If `age ≥ 18`

- The program would skip the throw statement and print "Eligible to vote".
-

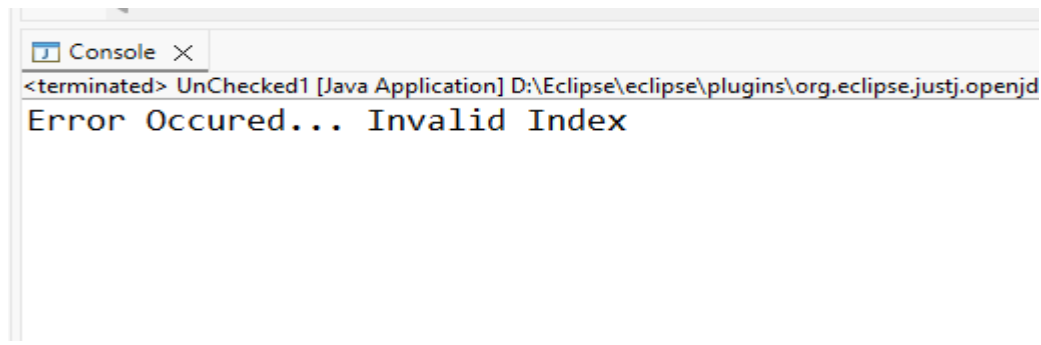
4. **UNCHECKED EXCEPTION:**

PROGRAM 1:

Unchecked exception (invalid array index)

```
public class UnChecked1 {
    public static void main(String[] args) {
        int[] numbers = {10, 20, 30};
        try
        {
            System.out.println(numbers[5]);
        }
        catch (Exception e)
        {
            System.out.println("Error Occured... Invalid Index ");
        }
    }
}
```

Output:



Class & Main Method

- public class UnChecked1 - defines your program.
- public static void main(String[] args) - entry point of the program.

Array Declaration

- int[] numbers = {10, 20, 30}; - creates an array with 3 elements.
- Valid indices are 0, 1, 2.

try block

- Inside try, the program attempts:

java

System.out.println(numbers[5]);

- But index 5 does not exist (array size is only 3).
- This causes an **ArrayIndexOutOfBoundsException**, which is an **Unchecked Exception**.

catch block

- The exception is caught by catch (Exception e).
- System.out.println("Error Occured... Invalid Index "); prints a friendly error message instead of crashing.

If index is valid

- Example: numbers[1] → program would print 20.
- No exception would be thrown.

PROGRAM 2:

Unchecked exception (division by zero)

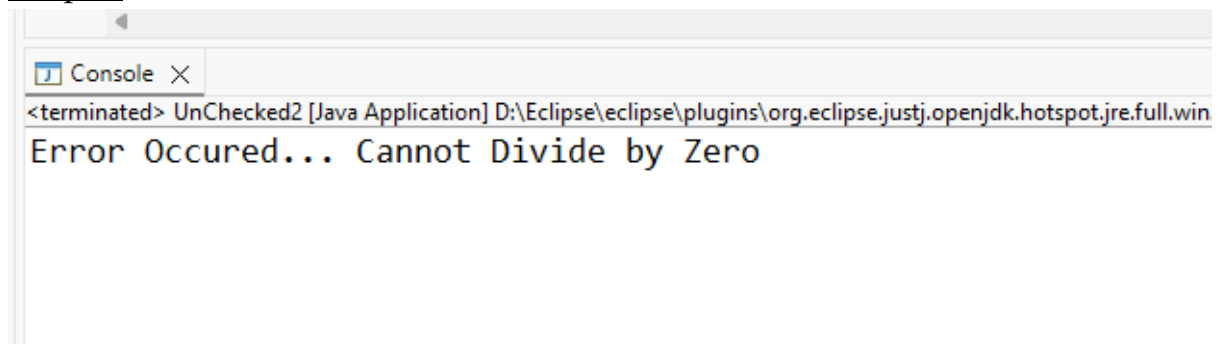
```
public class UnChecked2 {  
    public static void main(String[] args) {  
        int a = 10;
```

```

int b = 0;
try {
    int c = a / b;
    System.out.println(c);
} catch (Exception e) {
    System.out.println("Error Occured... Cannot Divide by Zero");
}
}
}

```

Output:



Class & Main Method

- public class UnChecked2 → defines your program.
- public static void main(String[] args) → entry point of the program.

Variables

- int a = 10; → variable a with value 10.
- int b = 0; → variable b with value 0. Division by zero is invalid.

try block

- Inside try, the program attempts:
int c = a / b;
- Since b = 0, dividing 10 by 0 causes an ArithmeticException.
- This is an Unchecked Exception (runtime error) because the compiler does not force you to handle it.

catch block

- The exception is caught by catch (Exception e).
- System.out.println("Error Occured... Cannot Divide by Zero"); prints a friendly error message instead of crashing.

If b ≠ 0

- Example: b = 2 → program would print 5.
- No exception would be thrown.